INDUSTRIAL INTERNSHIP PROJECT REPORT

MOTOR HEALTH MONITORING SYSTEM USING STM32F103C8T6

Submitted in partial fulfillment of the requirements for the completion of Internship Training

Submitted By

MADHUMITHA N
422423106034

Under the Guidance of

Mr. SILAMBARASAN

Technical Trainer

Ingage Technologies Pvt. Ltd.

Internship Organization

Ingage Technologies Pvt. Ltd.

Department of Electronics and Communication Engineering

UNIVERSITY COLLEGE OF ENGINEERING ,TINDIVANAM

(A Constituent College of Anna University)

Melpakkam, Tindivanam – 604 307
Villupuram District, Tamil Nadu

Academic Year

2025 – 2026

Abstract

Industrial motors play a vital role in manufacturing and automation systems, and their continuous operation makes them vulnerable to faults such as overheating, excessive vibration, and abnormal current consumption. If these issues are not identified at an early stage, they can lead to equipment damage, unexpected downtime, and increased maintenance costs.

This project presents a Motor Health Monitoring System developed using the STM32F103C8T6 microcontroller. The system continuously monitors three important motor parameters: temperature, vibration, and current. Sensor data is acquired through the ADC channels of the microcontroller and compared with predefined threshold values to determine the health condition of the motor.

Based on the monitored values, the system classifies the motor condition into Normal, Warning, or Critical Fault states. Visual indications are provided using LEDs, while audible alerts are generated through a buzzer to notify the operator of abnormal conditions. A push button is also incorporated to acknowledge and temporarily mute alarms when required. In addition, UART communication is used to display real-time sensor readings and motor status on a serial terminal for monitoring and analysis.

The developed system provides a simple, cost-effective, and reliable solution for real-time motor condition monitoring. By detecting faults at an early stage, it helps improve equipment reliability, reduce maintenance efforts, and support preventive maintenance practices in industrial environments.

Introduction

In industrial environments, electric motors are one of the most important components used in machines, production lines, and automation systems. Since these motors run continuously for long hours, they often face problems like overheating, vibration, and abnormal current consumption. If these issues are not detected early, it may lead to sudden breakdowns, machine failure, or even safety hazards.

To avoid these problems, a **Motor Health Monitoring System** is designed to continuously monitor key parameters like temperature, vibration, and current of the motor. The system helps in detecting abnormal conditions at an early stage and gives alerts when the values exceed the safe limit. This makes it easier to maintain the motor and reduce unexpected failures.

The main aim of this project is to improve the reliability and lifespan of industrial motors by using simple sensors and a microcontroller-based system. It provides real-time monitoring and helps in preventive maintenance instead of reactive maintenance. This project is very useful in industries where continuous motor operation is required and downtime should be minimized.

Overall, this system helps in improving efficiency, reducing maintenance cost, and ensuring safe operation of industrial motors.

Problem Statement – Motor Health Monitoring System

In many industrial applications, electric motors are operated continuously for long durations under different load conditions. During this operation, problems like overheating, excessive vibration, and abnormal current may occur. These issues usually develop slowly and are not easily noticed in the early stage.

In most cases, maintenance is done only after a failure happens, which leads to unexpected machine downtime, production loss, and increased repair cost. Sudden motor failure can also affect other connected systems and may create safety risks in the working environment.

So, there is a need for a simple and effective system that can continuously monitor the health condition of the motor and detect any abnormal behavior at an early stage. This will help in preventing major damage and support timely maintenance before failure occurs.

Objective

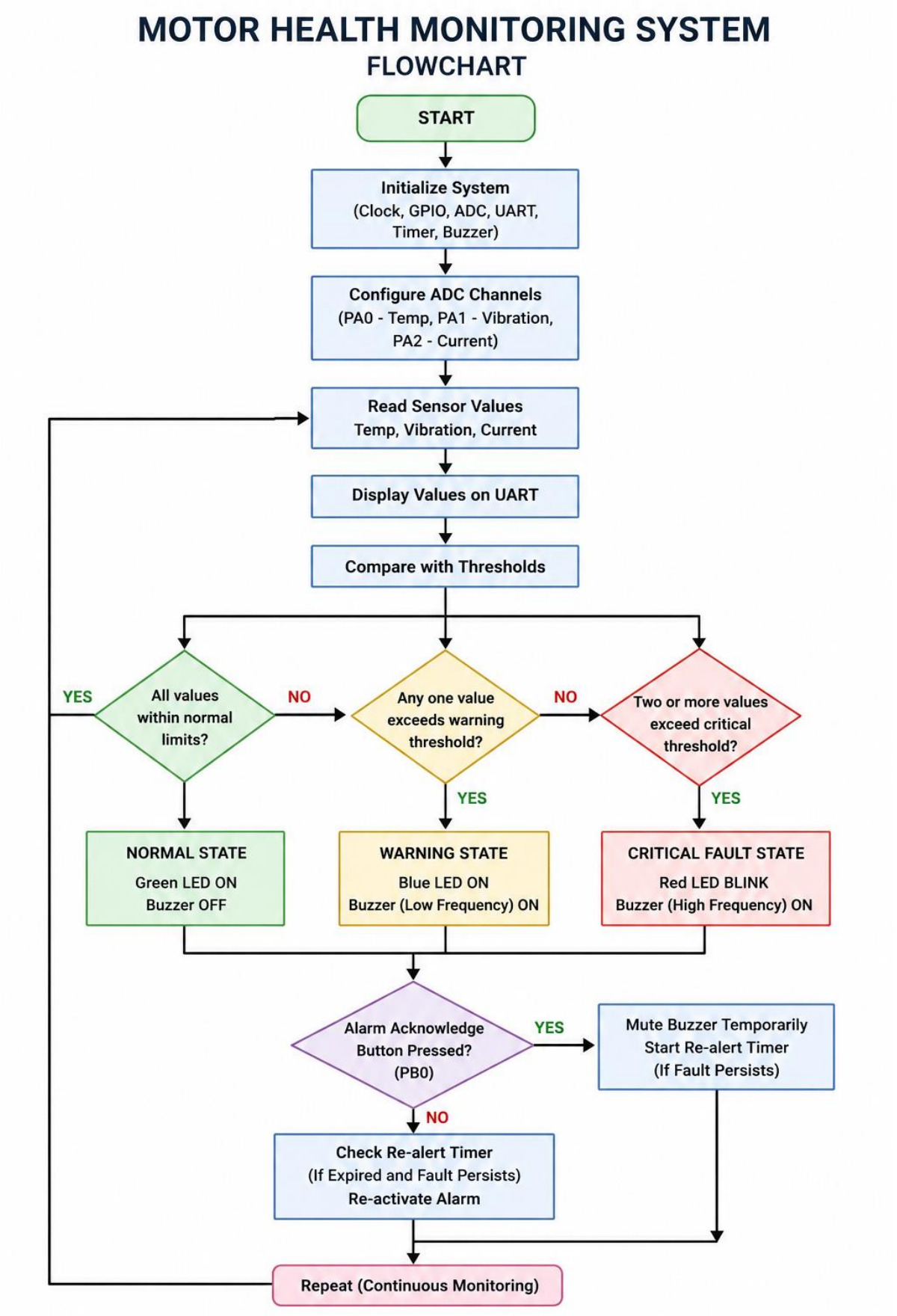
The main objective of this project is to design and develop a system that can continuously monitor the health condition of an industrial motor and detect faults at an early stage.

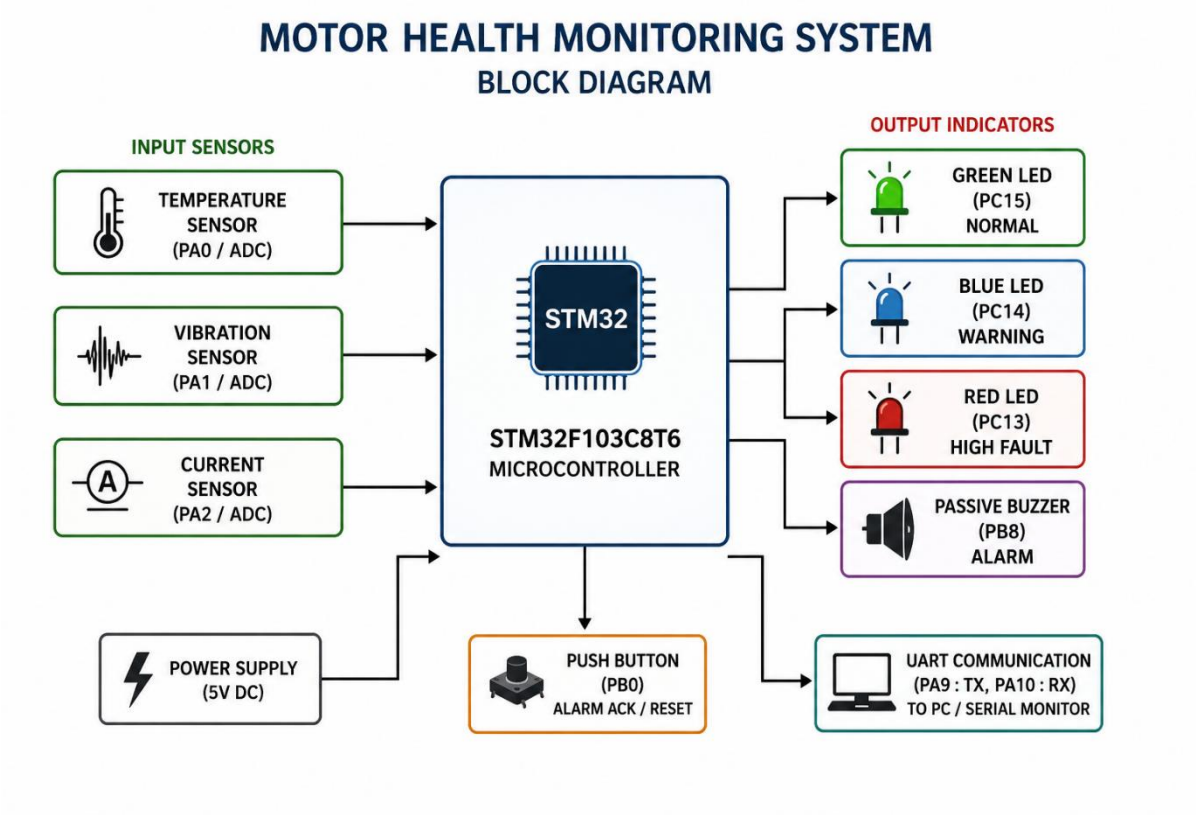
The specific objectives are:

- To monitor important motor parameters such as temperature, vibration, and current in real time.
- To detect abnormal conditions before they lead to major failure or damage.
- To provide alerts when any parameter exceeds the safe operating limit.
- To improve the reliability and lifespan of the motor through preventive maintenance.
- To reduce unexpected downtime and maintenance cost in industrial applications.
- To ensure safe and efficient operation of the motor under different working conditions.

Overall, this system aims to support industries by making motor maintenance smarter, faster, and more reliable.

Block Diagram

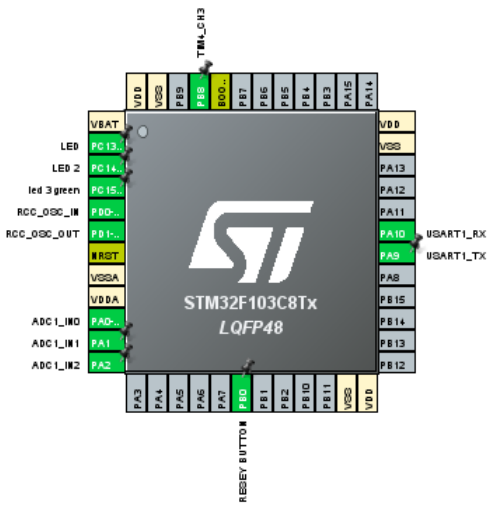




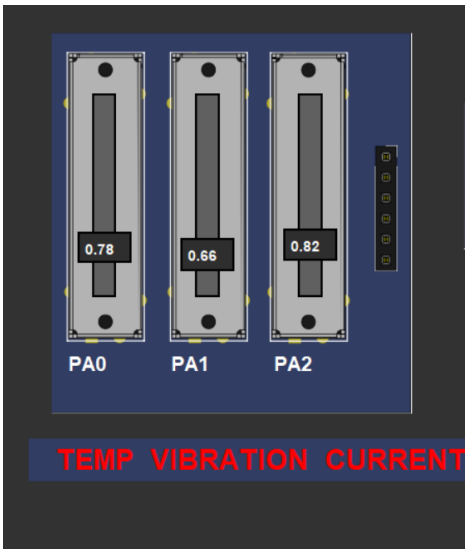
Components Requirement

1. Hardware / Simulation Components

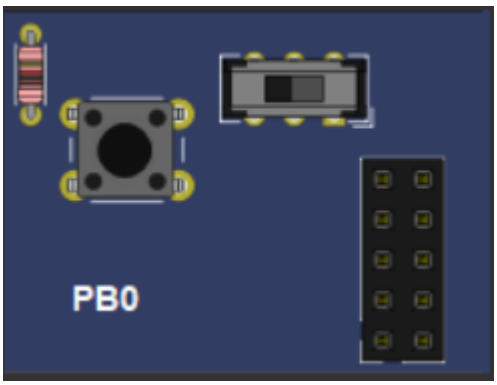
- **STM32F103C8T6 (Blue Pill)** – Main microcontroller used for processing sensor data and controlling outputs



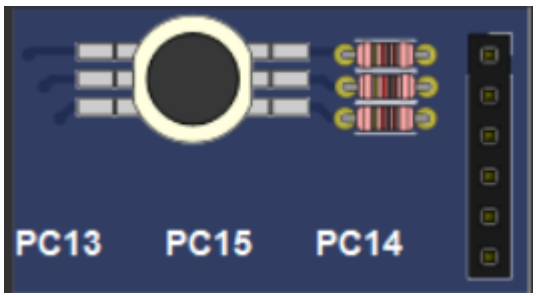
- **Potentiometer (x2 or x3)** – Used to simulate analog sensor inputs such as temperature, current, or vibration levels



- **Push Button Switch** – Used for reset, mode change, or fault simulation input

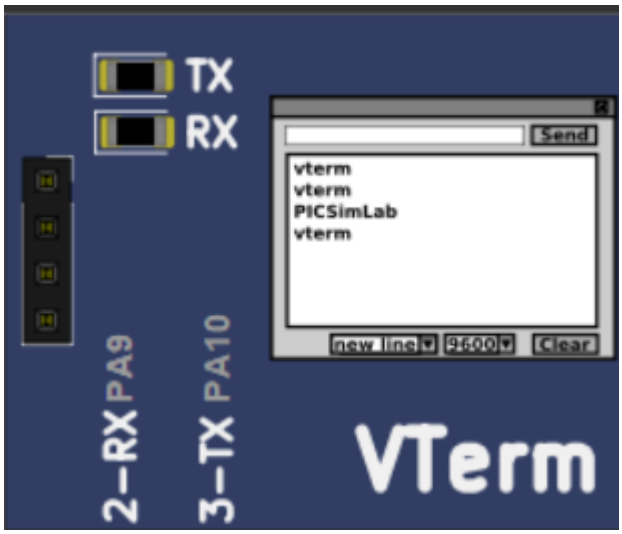


- **LEDs (Red, Green, Blue)** – Indication of motor condition (Normal, Warning, Fault)



- **Buzzer** – Provides audio alert during fault or abnormal condition

- **IO Virtual / PICSimLab Interface** – Used for connecting and testing virtual input/output devices in simulation



2. Software Requirements

- **STM32CubeIDE** – For coding, compiling, and debugging STM32 firmware
- **STM32CubeMX (inside CubeIDE)** – For GPIO, ADC, and peripheral configuration
- **PICSimLab Simulator** – For hardware simulation and testing

3. Simulation Inputs (Motor Condition Representation)

- Potentiometer 1 → Temperature sensor simulation
- Potentiometer 2 → Current sensor simulation
- Potentiometer 3 → Vibration sensor simulation

4. Output Indicators

- Green LED → Warning condition
- Red LED → Fault condition
- Blue LED→ More than 1 fault condition
- Buzzer → High priority fault alert

Working Principle

The Motor Health Monitoring System works by continuously monitoring the operating condition of a motor through input parameters that represent **temperature, current, and vibration levels**. In this project, these parameters are simulated using potentiometers in the PICSimLab environment.

The potentiometer values are read by the **STM32F103C8T6 microcontroller** through its Analog-to-Digital Converter (ADC). The microcontroller continuously compares the received values with predefined threshold limits to determine the motor's health condition.

When all parameter values are within the safe range, the system indicates a **normal operating condition** by turning ON the green LED. If any parameter reaches a warning level, the yellow LED is activated to notify the user that the motor requires attention. When the values exceed the critical limit, the system identifies it as a fault condition and activates the red LED along with the buzzer to provide both visual and audible alerts.

A push button is provided for user interaction, such as acknowledging alerts or resetting the system state. The entire monitoring process runs continuously in real time, allowing early detection of abnormal motor conditions and helping prevent unexpected motor failures.

Thus, the system provides a simple and effective method for monitoring motor health, improving reliability, reducing downtime, and supporting preventive maintenance in industrial applications.

Embedded C Program Flow

1. Initialize STM32 peripherals.
2. Configure ADC channels for sensors.
3. Configure GPIO pins for LEDs and buzzer.
4. Read temperature sensor value.
5. Read vibration sensor value.
6. Read current sensor value.
7. Compare sensor values with thresholds.
8. Determine motor health condition.
9. Activate LEDs and buzzer accordingly.
10. Display sensor readings through UART.
11. Check push button status.

12. Repeat monitoring process continuously.

EMBEDDED C SOURCE CODE

```
#include "main.h"

#include <stdio.h>

#include <string.h>


/* USER CODE BEGIN PTD */

typedef enum {

   STATE_NORMAL,

   STATE_INTERMEDIATE_WARNING,

   STATE_WARNING_ACKNOWLEDGED,

   STATE_HIGH_FAULT

} SystemState_t;

/* USER CODE END PTD */




/* Private variables ---------------------------------------------------------*/

ADC_HandleTypeDef hadc1;


TIM_HandleTypeDef htim4;


UART_HandleTypeDef huart1;


/* USER CODE BEGIN PV */

SystemState_t currentState = STATE_NORMAL;
```

```
/* Timing Trackers */
uint32_t ackTimerStart = 0;
uint32_t lastToggleTime = 0;
uint32_t lastPrintTime = 0;

/* Output tracking variables */
uint8_t toggleState = 0;
uint8_t ackButtonPressed = 0;
/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_ADC1_Init(void);
static void MX_TIM4_Init(void);
static void MX_USART1_UART_Init(void);
/* USER CODE BEGIN PFP */
uint32_t ADC_Read(uint32_t channel)
{
  ADC_ChannelConfTypeDef sConfig = {0};

  sConfig.Channel = channel;
  sConfig.Rank = ADC_REGULAR_RANK_1;
  sConfig.SamplingTime = ADC_SAMPLETIME_55CYCLES_5;
```

```
    HAL_ADC_ConfigChannel(&hadc1, &sConfig);

    HAL_ADC_Start(&hadc1);
    // Increased timeout for safer multisample reads
    HAL_ADC_PollForConversion(&hadc1, 100);

    uint32_t value = HAL_ADC_GetValue(&hadc1);

    HAL_ADC_Stop(&hadc1);

    return value;
}

void Buzzer_Start(uint16_t freq)
{
    uint32_t arr = (72000000 / freq) - 1;

    __HAL_TIM_SET_AUTORELOAD(&htim4, arr);
    __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_3, arr/2);
    HAL_TIM_PWM_Start(&htim4, TIM_CHANNEL_3);
}

void Buzzer_Stop(void)
{
```

```
    HAL_TIM_PWM_Stop(&htim4, TIM_CHANNEL_3);
    __HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_3, 0);
    // Explicit low fallback for passive piezo drivers on PB8
    HAL_GPIO_WritePin(GPIOB, GPIO_PIN_8, GPIO_PIN_RESET);
}

void Motor_Monitor(void)
{
    char buf[600];
    uint32_t currentTime = HAL_GetTick();

    /* 1. Sensor Acquisition & Mapping */
    uint8_t temp = ADC_Read(ADC_CHANNEL_0) * 100 / 4095;
    uint8_t vib  = ADC_Read(ADC_CHANNEL_1) * 100 / 4095;
    uint8_t curr = ADC_Read(ADC_CHANNEL_2) * 100 / 4095;
    uint8_t health = 100 - ((temp + vib + curr) / 3);

    /* 2. Flag Assertions */
    uint8_t temp_HF = (temp >= 84);
    uint8_t vib_HF  = (vib >= 84);
    uint8_t curr_HF = (curr >= 84);
    uint8_t highFaultCount = temp_HF + vib_HF + curr_HF;

    uint8_t temp_IW = (temp >= 70 && temp <= 83);
    uint8_t vib_IW  = (vib >= 70 && vib <= 83);
```

```
uint8_t curr_IW = (curr >= 70 && curr <= 83);


/*uint8_t anyHighFault = (temp_HF || vib_HF || curr_HF);*/
uint8_t anyIntermediateWarning = (temp_IW || vib_IW || curr_IW);


/* 3. Button Evaluation (PB0 - Active Low Configuration with Pull-up) */
if (HAL_GPIO_ReadPin(GPIOB, GPIO_PIN_0) == GPIO_PIN_RESET) {
   if (!ackButtonPressed) {
      ackButtonPressed = 1; // Debounce latch flag
      if (currentState == STATE_INTERMEDIATE_WARNING) {
         currentState = STATE_WARNING_ACKNOWLEDGED;
         ackTimerStart = currentTime;


         // Instantly silence indicators on acknowledgment
         HAL_GPIO_WritePin(GPIOC, GPIO_PIN_15, GPIO_PIN_RESET);
         Buzzer_Stop();
      }
   }
} else {
   ackButtonPressed = 0;
}



if(highFaultCount >= 2)
{
```

```
    currentState = STATE_HIGH_FAULT;
}
else if(highFaultCount == 1)
{
    currentState = STATE_HIGH_FAULT;
}
else if(anyIntermediateWarning)
{
    if(currentState == STATE_WARNING_ACKNOWLEDGED)
    {
        if(currentTime >= (ackTimerStart + 30000))
        {
            currentState = STATE_INTERMEDIATE_WARNING;
        }
    }
    else
    {
        currentState = STATE_INTERMEDIATE_WARNING;
    }
}
else
{
    currentState = STATE_NORMAL;
}
```

```
/* 5. State Execution Logic */
switch (currentState)
{
   case STATE_NORMAL:
      HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET);
      HAL_GPIO_WritePin(GPIOC, GPIO_PIN_14, GPIO_PIN_RESET);
      HAL_GPIO_WritePin(GPIOC, GPIO_PIN_15, GPIO_PIN_RESET);

      Buzzer_Stop();

      break;

   case STATE_INTERMEDIATE_WARNING:
      HAL_GPIO_WritePin(GPIOC, GPIO_PIN_14, GPIO_PIN_RESET); // Blue OFF
      HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET); // Red OFF

      // Synchronized Non-blocking Blinking Pattern (500ms Duty Interval)
      if (currentTime - lastToggleTime >= 500) {
         lastToggleTime = currentTime;
         toggleState = !toggleState;

         HAL_GPIO_WritePin(GPIOC, GPIO_PIN_15, toggleState ? GPIO_PIN_SET :
GPIO_PIN_RESET);
         if (toggleState) {
            Buzzer_Start(1200); // Distinct Intermediate warning beep pitch
```

```
    } else {
        Buzzer_Stop();
    }
}

// Print intermediate warning block once every 4 seconds to limit UART spam
if (currentTime - lastPrintTime >= 4000) {
    lastPrintTime = currentTime;
    char tempMsg[64] = "";
    char vibMsg[64] = "";
    char currMsg[64] = "";

    if (temp_IW) strcpy(tempMsg, "- TEMP: INTERMEDIATE OVERHEATING (CHECK FAN)\r\n");
    if (vib_IW)  strcpy(vibMsg,  "- VIB: INTERMEDIATE VIBRATION (INSPECT BEARING)\r\n");
    if (curr_IW) strcpy(currMsg, "- CURR: INTERMEDIATE OVERLOAD (REDUCE LOAD)\r\n");

    sprintf(buf,
        "\r\n============================\r\n"
        "INTERMEDIATE WARNING ACTIVE\r\n"
        "============================\r\n"
        "TEMP   : %d %%\r\n"
        "VIB    : %d %%\r\n"
        "CURR   : %d %%\r\n"
```

```
            "HEALTH : %d %%\r\n\r\n"
            "WARNING DETAILS:\r\n%s%s%s\r\n"
            "MAINTENANCE REQUIRED\r\n"
            "============================\r\n",
            temp, vib, curr, health, tempMsg, vibMsg, currMsg);
        HAL_UART_Transmit(&huart1, (uint8_t*)buf, strlen(buf), 200);
    }
    break;

case STATE_WARNING_ACKNOWLEDGED:
    // Warnings remain muted during acknowledgment period
    HAL_GPIO_WritePin(GPIOC, GPIO_PIN_14, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOC, GPIO_PIN_15, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET);
    Buzzer_Stop();
    break;

case STATE_HIGH_FAULT:
     HAL_GPIO_WritePin(GPIOC, GPIO_PIN_15, GPIO_PIN_RESET);
    if(highFaultCount >= 2)
    {
        HAL_GPIO_WritePin(GPIOC, GPIO_PIN_14, GPIO_PIN_SET);
        Buzzer_Start(4000);
    }
    else
```

```
{
    HAL_GPIO_WritePin(GPIOC, GPIO_PIN_14, GPIO_PIN_RESET);
    Buzzer_Start(1800);
}

// Red LED Blinking (Fast 200ms Interval)
if (currentTime - lastToggleTime >= 200) {
    lastToggleTime = currentTime;
    HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);
}

// Print High Fault updates every 2 seconds
if (currentTime - lastPrintTime >= 2000) {
    lastPrintTime = currentTime;
    char fMsg[128] = "";
    if (temp_HF) strcat(fMsg, "- CRITICAL TEMPERATURE FAULT!\r\n");
    if (vib_HF)  strcat(fMsg, "- CRITICAL VIBRATION EXCESS!\r\n");
    if (curr_HF) strcat(fMsg, "- CRITICAL MOTOR OVERLOAD!\r\n");

    sprintf(buf,
        "\r\n!!!!!!!!!!!!!!!!!!!!!!!!!!!!\r\n"
        "       CRITICAL HIGH FAULT\r\n"
        "!!!!!!!!!!!!!!!!!!!!!!!!!!!!\r\n"
        "TEMP   : %d %%\r\n"
        "VIB    : %d %%\r\n"
```

```
                "CURR   : %d %%\r\n"
                "HEALTH : %d %%\r\n\r\n"
                "EMERGENCY FAULT LOG:\r\n%s\r\n"
                "ACTION REQUIRED IMMEDIATELY!\r\n"
                "!!!!!!!!!!!!!!!!!!!!!!!!!!!!\r\n",
                temp, vib, curr, health, fMsg);
            HAL_UART_Transmit(&huart1, (uint8_t*)buf, strlen(buf), 200);
        }
        break;
    }
}

int main(void)
{
 /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
  HAL_Init();
 /* Configure the system clock */
  SystemClock_Config();
/* Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_ADC1_Init();
  MX_TIM4_Init();
  MX_USART1_UART_Init();
  /* USER CODE BEGIN 2 */
  HAL_TIM_PWM_Start(&htim4, TIM_CHANNEL_3);
```

```
 HAL_TIM_PWM_Stop(&htim4, TIM_CHANNEL_3);
 /* USER CODE END 2 */
 while (1)
          {
             Motor_Monitor();
             HAL_Delay(50);
          }
}
void SystemClock_Config(void)
{
 RCC_OscInitTypeDef RCC_OscInitStruct = {0};
 RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
 RCC_PeriphCLKInitTypeDef PeriphClkInit = {0};

 /** Initializes the RCC Oscillators according to the specified parameters
 * in the RCC_OscInitTypeDef structure.
 */
 RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
 RCC_OscInitStruct.HSEState = RCC_HSE_ON;
 RCC_OscInitStruct.HSEPredivValue = RCC_HSE_PREDIV_DIV1;
 RCC_OscInitStruct.HSIState = RCC_HSI_ON;
 RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
 RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
 RCC_OscInitStruct.PLL.PLLMUL = RCC_PLL_MUL9;
 if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
```

```
  {
    Error_Handler();
  }

  /** Initializes the CPU, AHB and APB buses clocks
  */
  RCC_ClkInitStruct.ClockType =
RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV2;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_2) != HAL_OK)
  {
    Error_Handler();
  }
  PeriphClkInit.PeriphClockSelection = RCC_PERIPHCLK_ADC;
  PeriphClkInit.AdcClockSelection = RCC_ADCPCLK2_DIV6;
  if (HAL_RCCEx_PeriphCLKConfig(&PeriphClkInit) != HAL_OK)
  {
    Error_Handler();
  }
}
```

```
/**
  * @brief ADC1 Initialization Function
  * @param None
  * @retval None
  */
static void MX_ADC1_Init(void)
{
  /* USER CODE BEGIN ADC1_Init 0 */
/* USER CODE END ADC1_Init 0 */
ADC_ChannelConfTypeDef sConfig = {0};
/* USER CODE BEGIN ADC1_Init 1 */
 /* USER CODE END ADC1_Init 1 */
/** Common config*/
  hadc1.Instance = ADC1;
  hadc1.Init.ScanConvMode = ADC_SCAN_DISABLE;
  hadc1.Init.ContinuousConvMode = DISABLE;
  hadc1.Init.DiscontinuousConvMode = DISABLE;
  hadc1.Init.ExternalTrigConv = ADC_SOFTWARE_START;
  hadc1.Init.DataAlign = ADC_DATAALIGN_RIGHT;
  hadc1.Init.NbrOfConversion = 1;
  if (HAL_ADC_Init(&hadc1) != HAL_OK)
  {
    Error_Handler();
  }
```

```
  /* Configure Regular Channel */
  sConfig.Channel = ADC_CHANNEL_0;
  sConfig.Rank = ADC_REGULAR_RANK_1;
  sConfig.SamplingTime = ADC_SAMPLETIME_1CYCLE_5;
  if (HAL_ADC_ConfigChannel(&hadc1, &sConfig) != HAL_OK)
  {
    Error_Handler();
  }
}
static void MX_TIM4_Init(void)
{
 TIM_MasterConfigTypeDef sMasterConfig = {0};
 TIM_OC_InitTypeDef sConfigOC = {0};
  htim4.Instance = TIM4;
  htim4.Init.Prescaler = 0;
  htim4.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim4.Init.Period = 65535;
  htim4.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim4.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
  if (HAL_TIM_PWM_Init(&htim4) != HAL_OK)
  {
    Error_Handler();
  }
  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
```

```
  if (HAL_TIMEx_MasterConfigSynchronization(&htim4, &sMasterConfig) != HAL_OK)
  {
    Error_Handler();
  }
  sConfigOC.OCMode = TIM_OCMODE_PWM1;
  sConfigOC.Pulse = 0;
  sConfigOC.OCPolarity = TIM_OCPOLARITY_HIGH;
  sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
  if (HAL_TIM_PWM_ConfigChannel(&htim4, &sConfigOC, TIM_CHANNEL_3) != HAL_OK)
  {
    Error_Handler();
  }
  HAL_TIM_MspPostInit(&htim4);
}
static void MX_USART1_UART_Init(void)
{
  huart1.Instance = USART1;
  huart1.Init.BaudRate = 115200;
  huart1.Init.WordLength = UART_WORDLENGTH_8B;
  huart1.Init.StopBits = UART_STOPBITS_1;
  huart1.Init.Parity = UART_PARITY_NONE;
  huart1.Init.Mode = UART_MODE_TX_RX;
  huart1.Init.HwFlowCtl = UART_HWCONTROL_NONE;
  huart1.Init.OverSampling = UART_OVERSAMPLING_16;
```

```
  if (HAL_UART_Init(&huart1) != HAL_OK)
  {
    Error_Handler();
  }
}
static void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};

  __HAL_RCC_GPIOC_CLK_ENABLE();
  __HAL_RCC_GPIOD_CLK_ENABLE();
  __HAL_RCC_GPIOA_CLK_ENABLE();
  __HAL_RCC_GPIOB_CLK_ENABLE();

  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(GPIOC, LED_Pin|LED_2_Pin|led_3_green_Pin,
GPIO_PIN_RESET);

  /*Configure GPIO pins : LED_Pin LED_2_Pin led_3_green_Pin */
  GPIO_InitStruct.Pin = LED_Pin|LED_2_Pin|led_3_green_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);
  /*Configure GPIO pin : RESEY_BUTTON_Pin */
```

```
  GPIO_InitStruct.Pin = RESEY_BUTTON_Pin;
  GPIO_InitStruct.Mode = GPIO_MODE_INPUT;
  GPIO_InitStruct.Pull = GPIO_PULLUP;
  HAL_GPIO_Init(RESEY_BUTTON_GPIO_Port, &GPIO_InitStruct);
}
void Error_Handler(void)
{
  __disable_irq();
  while (1)
  {
  }
  /* USER CODE END Error_Handler_Debug */
}
#ifdef USE_FULL_ASSERT
void assert_failed(uint8_t *file, uint32_t line)
{
}
#endif /* USE_FULL_ASSERT */
```

Communication Protocol Used / Simulation Output

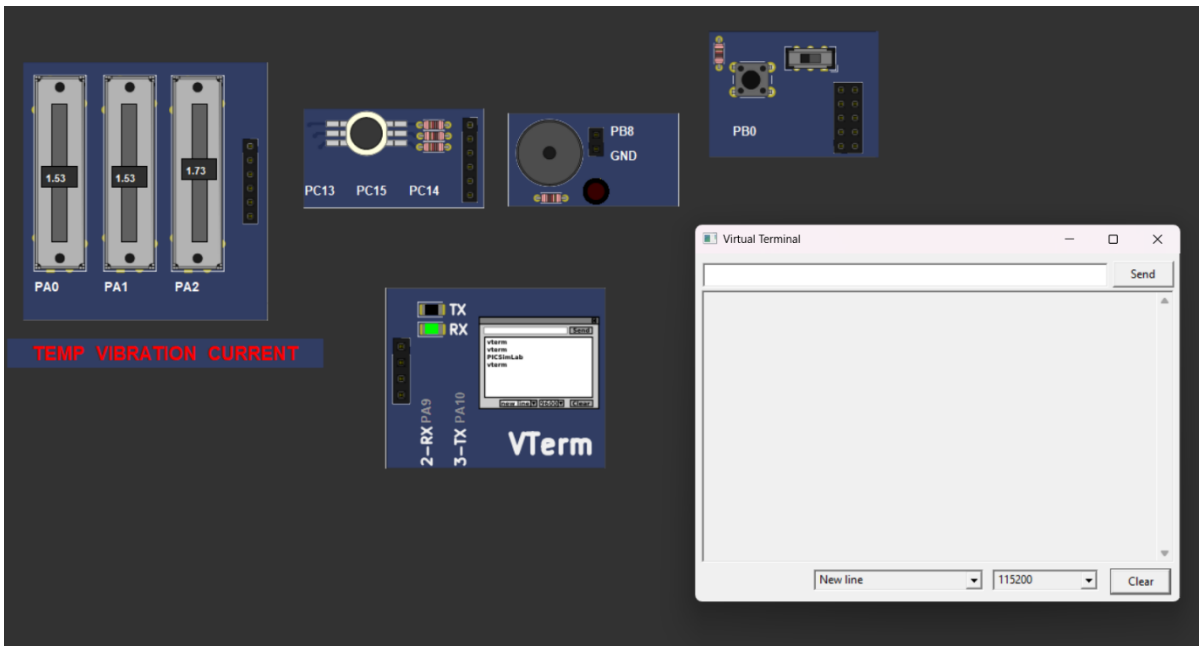
Communication Protocol Used: UART (Universal Asynchronous Receiver Transmitter)

UART communication is used to transmit real-time motor health data from the STM32F103C8T6 microcontroller to the serial terminal available in PICSimLab. This enables continuous monitoring of motor parameters and helps in debugging the system during simulation.

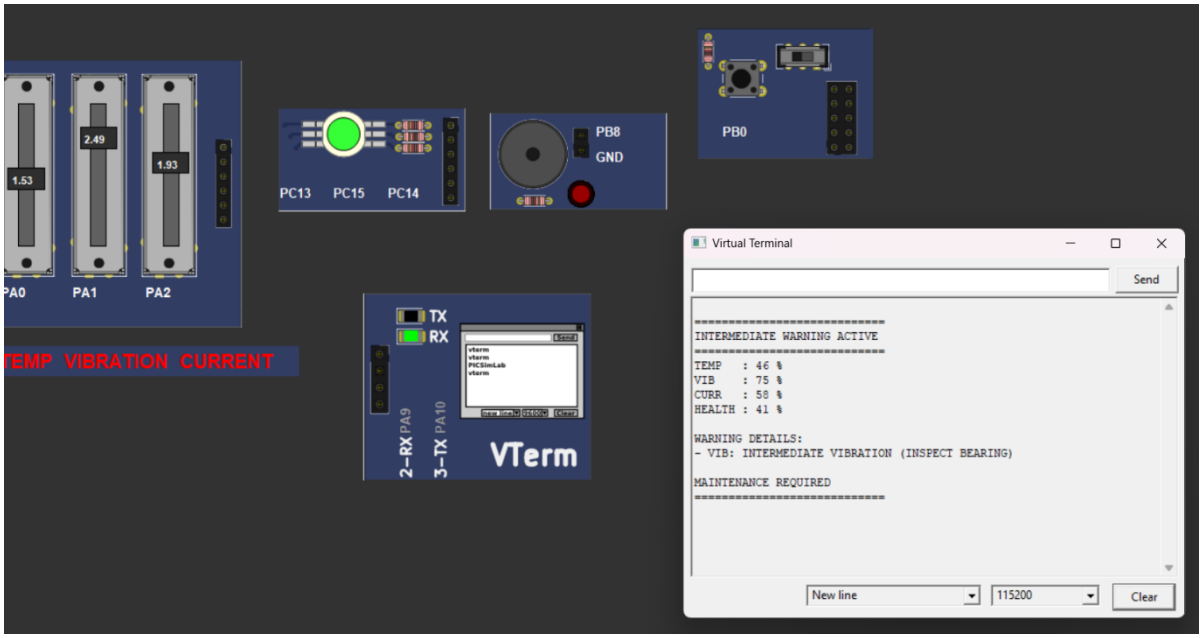
The microcontroller reads the simulated sensor values (temperature, vibration, and current) and sends the corresponding motor status through UART at regular intervals.

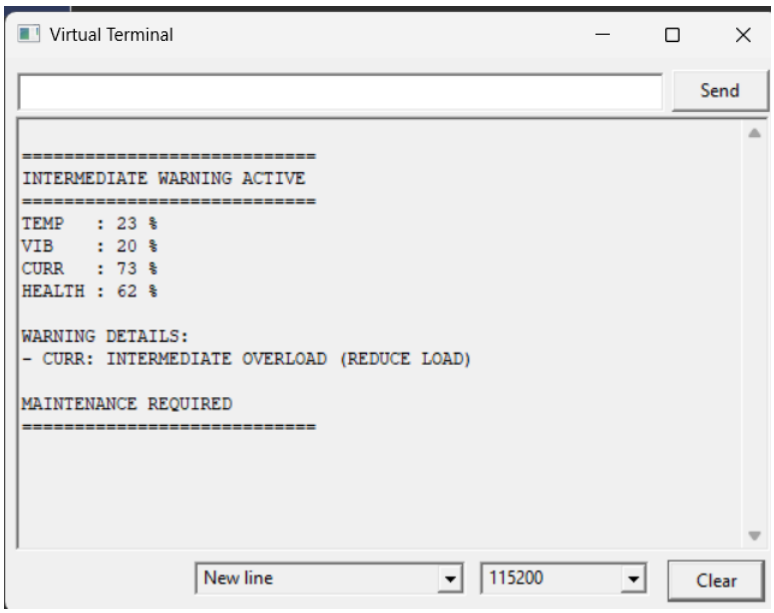
Sample Simulation Output

Normal Condition



Warning Condition



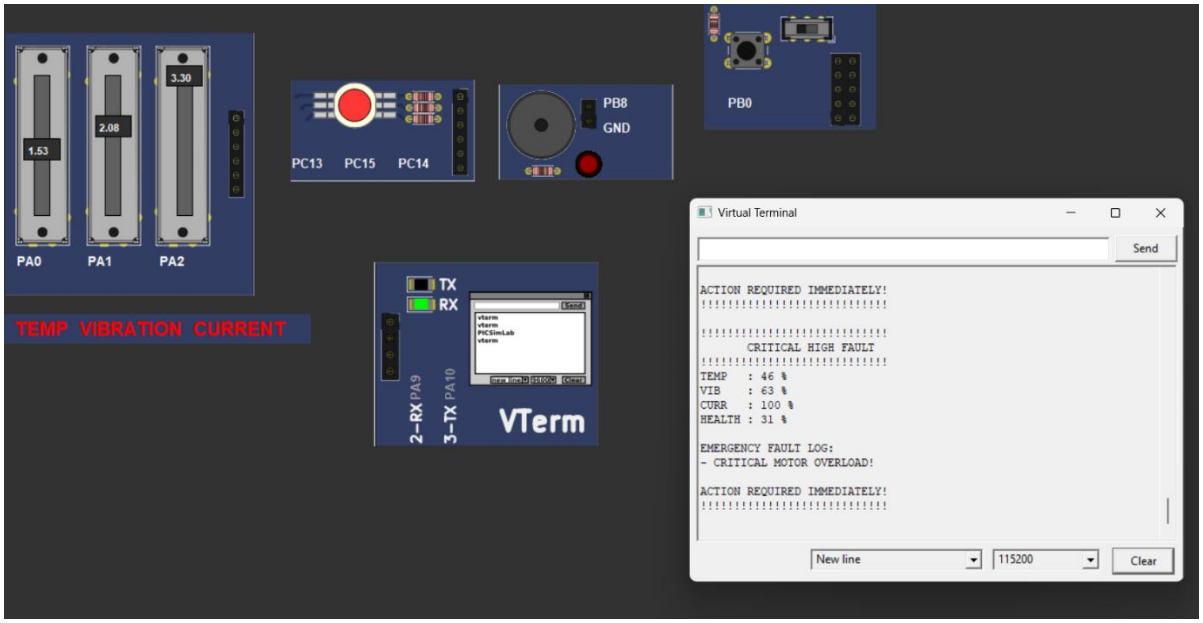


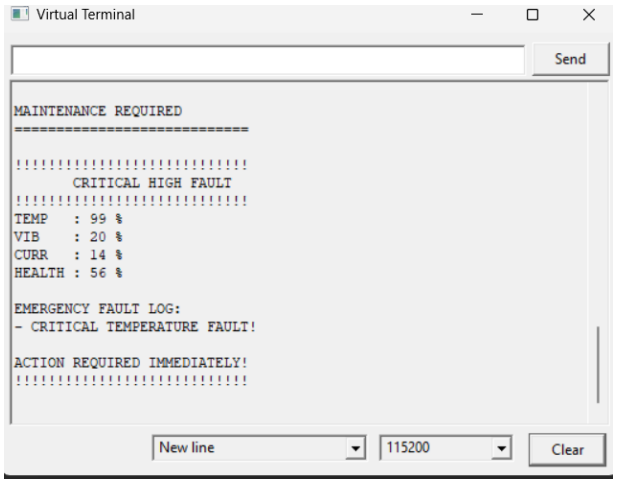
BUZZER Activated

GREEN LED ON

Pressing the **push button** mutes the warning for **30 seconds**; if the condition persists, the warning automatically reappears.

Critical Fault Condition



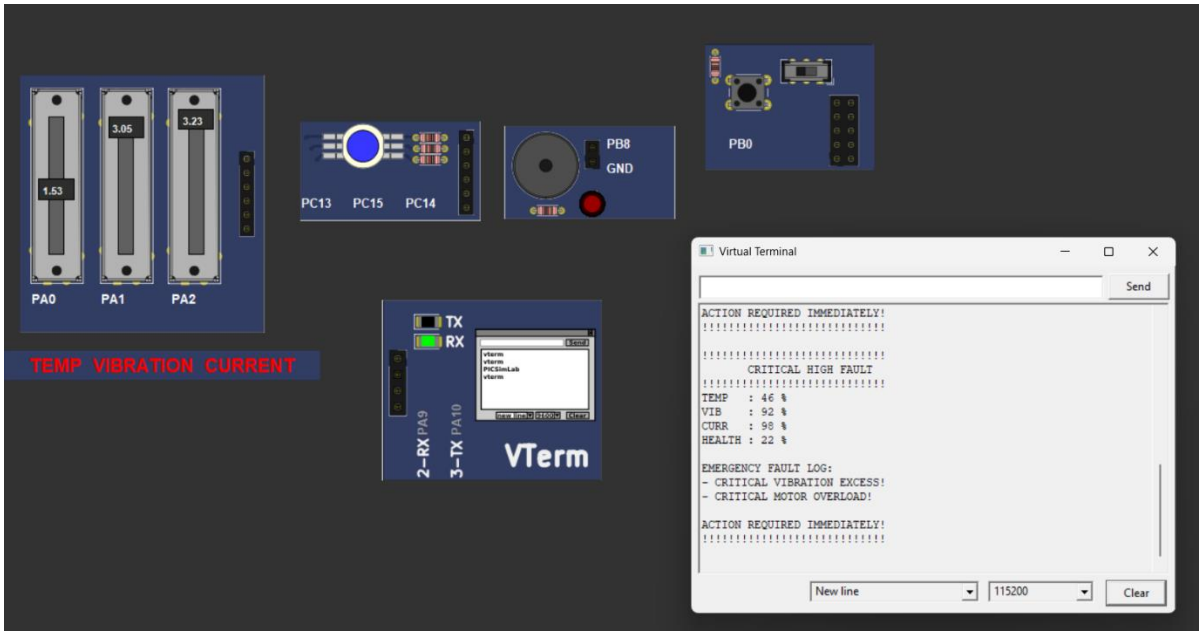


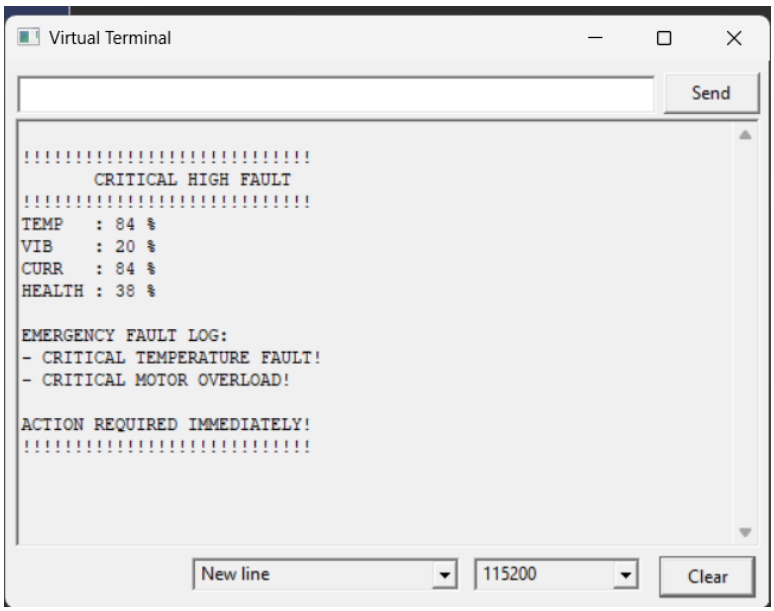
Buzzer Activated

Red LED ON

Not NEED PUSH BUTTON

More than 1 High Fault





Buzzer Activated

Blue LED ON

Not NEED PUSH BUTTON

Status Indications

Motor Condition	Green LED	Red LED	Blue LED	Buzzer
Normal	OFF	OFF	OFF	OFF
Intermediate Warning	Blinking	OFF	OFF	1200 Hz Beep
High Fault (1 Parameter Critical)	OFF	Blinking	OFF	1800 Hz
Severe Fault (2+ Parameters Critical)	OFF	Blinking	ON	4000 Hz

OBSERVATION

During the simulation, the STM32F103C8T6 continuously monitored the values of temperature, vibration, and current through analog inputs. The system successfully classified the motor condition into different operating states based on predefined threshold values.

When all parameters remained below 70%, the motor operated in the Normal State and no warning indicators were activated. When any parameter entered the range of 70% to 83%, the system entered the Intermediate Warning State, where the Green LED blinked and the buzzer generated periodic warning sounds. The warning could be temporarily acknowledged using the push button.

When one parameter exceeded 84%, the system entered the High Fault State. In this condition, the Red LED blinked continuously and the buzzer produced a higher-frequency alert sound. If two or more parameters simultaneously exceeded the critical threshold, the Blue LED was activated to indicate a severe fault condition requiring immediate maintenance.

All sensor readings, health percentage values, warning messages, and fault information were successfully transmitted to the serial terminal through UART communication in real time.

RESULT

The Industrial Motor Health Monitoring System was successfully designed and simulated using STM32F103C8T6, STM32CubeIDE, and PICSimLab. The system continuously monitored motor health parameters such as temperature, vibration, and current, and accurately detected abnormal operating conditions.

The implemented warning and fault detection mechanism successfully provided visual indications using Green, Red, and Blue LEDs, along with audible alerts through a buzzer. UART communication enabled real-time monitoring and fault reporting through the serial terminal.

The project effectively demonstrated early fault detection, condition monitoring, and preventive maintenance concepts. The developed system can help reduce unexpected motor failures, improve operational reliability, and minimize industrial downtime.

CONCLUSION

The Industrial Motor Health Monitoring System was successfully developed and simulated using the STM32F103C8T6 microcontroller, STM32CubeIDE, and PICSimLab. The system continuously monitored key motor parameters such as temperature, vibration, and current to evaluate the overall health condition of the motor.

Based on the sensor values, the system accurately identified Normal, Intermediate Warning, and High Fault conditions. Visual alerts were provided through Green, Red, and Blue LEDs, while audible alerts were generated using a buzzer. In addition, UART communication enabled real-time monitoring by displaying motor status, health percentage, and fault information on the serial terminal.

The project demonstrates an effective approach for early fault detection and preventive maintenance of industrial motors. By identifying abnormal operating conditions before major failures occur, the system can help reduce maintenance costs, minimize unexpected downtime, improve motor reliability, and enhance industrial safety.

Overall, the developed Motor Health Monitoring System provides a simple, cost-effective, and reliable solution for monitoring motor performance and supporting efficient maintenance practices in industrial environments.

REFERENCES

[1] STMicroelectronics, “STM32F103x8/STM32F103xB Datasheet,” 2024.

[2] STMicroelectronics, “STM32CubeIDE User Guide,” 2024.

[3] STMicroelectronics, “STM32 HAL Driver Documentation,” 2024.

[4] PICSimLab Documentation, “PICSimLab Simulator User Manual,” 2024.

[5] J. Axelson, Serial Port Complete, 2nd ed., Lakeview Research, 2007.

[6] R. C. Dorf and J. A. Svoboda, Introduction to Electric Circuits, Wiley, 2020.